

Debugging the Removals System

Mindaugas Taujanskas

I. REGISTERING SERVICE-PROVIDER CAUSES

USERALREADYEXISTSError

Upon attempting to register a company, an initial error is raised:

```
psycopg2.errors.CheckViolation: new row for relation
    "businesses" violates check constraint "
    chk_businesses__utr_no"
DETAIL:  Failing row contains (1, example business
    name, asdasdas, , , 123).
```

This stems from the following Python code which calls to register a business for a registered user:

```
user = self.register_user_from_detail_form(
    service_provider_details,
    "service-provider"
)
business = Business.create_for_user(
    user.token,
    business_name=form_data['business-name'],
    crn_no=form_data['crn'],
    vat_no=form_data['vat-number'],
    utr_no=form_data['utr-number'],
    num_employees=form_data['nr-employees']
)
```

It should be noted that the VAT/UTR fields are specified to be optional by the business requirements, however the relevant SQL check-constraint `chk_businesses__utr_no` (and also the VAT) are implemented as such:

```
CREATE TABLE Businesses (
    ...
    vat_no          TEXT UNIQUE,
    utr_no          TEXT UNIQUE,
    ...
    CONSTRAINT CHK_Businesses__vat_length
    CHECK (LENGTH(vat_no) = 11),
    CONSTRAINT CHK_Businesses__utr_no
    CHECK (LENGTH(utr_no) = 10)
);
```

This is contrary to the business requirements, and thus causes the Python code to fail in two ways. Firstly, the fields are notated `UNIQUE`, and so they cannot be `NULL`, as more than one duplicate value would immediately fail. Secondly, the check-constraint doesn't account for the fact either field could be empty. This can be ameliorated with the following changes:

```
CREATE TABLE Businesses (
    ...
    vat_no          TEXT,
    utr_no          TEXT,
    ...
    CONSTRAINT CHK_Businesses__vat_length
    CHECK (COALESCE(LENGTH(vat_no) = 11, TRUE)),
    CONSTRAINT CHK_Businesses__utr_no
    CHECK (COALESCE(LENGTH(utr_no) = 10, TRUE))
);
```

This removes the uniqueness constraint, and allows the field to either be empty, or text with length 11/10 respectively. The

`UNIQUE` constraint can be enforced at trigger-level on pre-insert onto the businesses table.

The second error is caused when the user attempts to register again with the same details:

```
File "controllers\role_selection.py", line 105, in
    register_user_from_detail_form
    user = register_user(**{
        "forename": self.user_auth['forename'],
        ...<4 lines>...
        "role": role
    })
File "models\user.py", line 71, in register_user
    data = db_errors.unwrap_result(db.
        proc_register_user(**details))
File "models\db_errors.py", line 99, in
    unwrap_result
    raise_from_error_t(result.error)
    ~~~~~
File "models\db_errors.py", line 77, in
    raise_from_error_t
    raise exc_cls(f"Result returned error: {error.
        message!r}")
removals_system.exceptions.auth_exceptions.
    UserAlreadyExistsError: Result returned error: '
    Email already exists'
```

This follows from the code given before:

```
user = self.register_user_from_detail_form(
    service_provider_details,
    "service-provider"
)
business = Business.create_for_user(
    user.token,
    business_name=form_data['business-name'],
    crn_no=form_data['crn'],
    vat_no=form_data['vat-number'],
    utr_no=form_data['utr-number'],
    num_employees=form_data['nr-employees']
)
```

The registration always succeeds, but the business creation doesn't, so then if the user attempts to move forward again after fixing their details, it will attempt to recreate the account. This can be solved in a couple of ways:

- 1) Roll-back the user creation if the business creation fails
- 2) Ensure the database logic is correct as to prevent this edge-case
- 3) Persist the user-token so that registration is not repeated in the same form

The simplest method we will apply is (2), however note that this falls short of addressing the real issue of weak state management. Also note that the `Business.create_for_user` call should be rewritten to:

```
business = Business.create_for_user(
    user.token,
    business_name=form_data['business-name'],
    crn_no=form_data['crn'],
    vat_no=form_data['vat-number'] or None,
    utr_no=form_data['utr-number'] or None,
    num_employees=form_data['nr-employees']
)
```

As an empty string is not equivalent to `NULL`. After applying these changes, the code works as intended.